

RAG-GFM: Overcoming In-Memory Bottlenecks in Graph Foundation Models via Retrieval-Augmented Generation

Haonan Yuan
SKLCCSE, School of Computer
Science and Engineering
Beihang University
Beijing, China
yuanhn@buaa.edu.cn

Qingyun Sun
SKLCCSE, School of Computer
Science and Engineering
Beihang University
Beijing, China
sunqy@buaa.edu.cn

Jiacheng Tao
SKLCCSE, School of Computer
Science and Engineering
Beihang University
Beijing, China
jiachengtao@buaa.edu.cn

Xingcheng Fu
Key Lab of Education Blockchain and
Intelligent Technology, Ministry of Education
Guangxi Normal University
Guilin, Guangxi, China
fuxc@gxnu.edu.cn

Jianxin Li*
SKLCCSE, School of Computer
Science and Engineering
Beihang University
Beijing, China
lijx@buaa.edu.cn

Abstract

Graph Foundation Models (GFM) have emerged as a frontier in graph learning, which are expected to deliver transferable representations across diverse tasks. However, GFM remain constrained by *in-memory bottlenecks*: they attempt to encode knowledge into model parameters, which limits semantic capacity, introduces heavy lossy compression with conflicts, and entangles graph representation with the knowledge in ways that hinder efficient adaptation, undermining scalability and interpretability. In this work, we propose **RAG-GFM**, a **R**etrieval-**A**ugmented **G**eneration aided **G**raph **F**oundation **M**odel that offloads knowledge from parameters and complements parameterized learning. To externalize graph knowledge, we build a dual-modal unified retrieval module, where a semantic store from prefix-structured text and a structural store from centrality-based motif. To preserve heterogeneous information, we design a dual-view alignment objective that contrasts both modalities to capture both content and relational patterns. To enable efficient downstream adaptation, we perform in-context augmentation to enrich supporting instances with retrieved texts and motifs as contextual evidence. Extensive experiments on five benchmark graph datasets demonstrate that RAG-GFM consistently outperforms 13 state-of-the-art baselines in both cross-domain node and graph classification, achieving superior effectiveness and efficiency.

CCS Concepts

• **Mathematics of computing** → **Graph algorithms**; • **Computing methodologies** → **Neural networks**; **Learning latent representations**; **Knowledge representation and reasoning**.

*Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792139>

Keywords

Graph Foundation Models, Retrieval-Augmented Generation, Multi-domain Graph Pre-training, Graph Prompt Learning

ACM Reference Format:

Haonan Yuan, Qingyun Sun, Jiacheng Tao, Xingcheng Fu, and Jianxin Li. 2026. RAG-GFM: Overcoming In-Memory Bottlenecks in Graph Foundation Models via Retrieval-Augmented Generation. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792139>

1 Introduction

Graphs are powerful structures for describing complex relationships among entities, and have been broadly adopted in domains such as modeling for the World Wide Web [1, 50], social and citation networks [7, 76], retrieval and recommendation systems [62, 65], knowledge graphs [63, 79], biological analysis [9, 67], etc. Graph Neural Networks (GNNs) [12, 20] have enabled effective representation learning on graphs, supporting a wide range of tasks, but are typically tailored to specific datasets and tasks, limiting cross-domain generalization. Motivated by large-scale pre-training in language and vision, Graph Foundation Models (GFM) have recently emerged to learn universal graph representations through pre-training and downstream adaptation [11, 14, 29, 34, 43, 44, 77], aiming to support diverse applications with minimal supervision.

Despite these advances, existing GFM face fundamental limitations. Current methods follow the “pretrain-then-finetune” paradigm overwhelmingly, where knowledge from source domains is either fully compressed into a single GFM’s model parameters [74, 81], or at best expanded through lightweight mixtures-of-experts (MoE) that remain largely conceptual and offer little practical relief [10, 78]. While parameter counts may increase marginally, they fall far short of matching the vast, orders-of-magnitude greater knowledge volume inherent in pre-training domains. Graph knowledge is inherently dual-modal, combining node-level semantic texts and higher-order structural patterns, which leads to inevitable *in-memory bottlenecks* that hinder scalability, robustness, and interpretability.

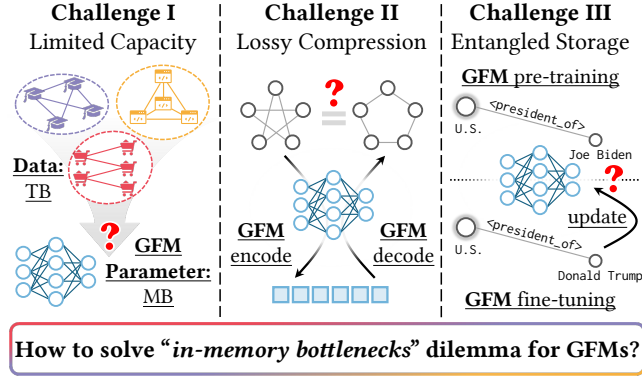


Figure 1: Challenges of the “in-memory bottlenecks”.

Challenge I: Limited capacity within parameters. Graph knowledge spans rich information whose scale far exceeds what model parameters can store. In graph models, increasing parameters or depth often causes over-smoothing rather than higher capacity [3, 18]. Consequently, GFMs trained on a single domain quickly exhaust their parameter budget when transferred to others with distinct semantics and motifs, leading to forgetting, poor transfer, and limited scalability [41, 82]. This exposes the fundamental limitation of parameter-centric storage for graphs.

Challenge II: Lossy and conflicting compression. Forcing heterogeneous graph knowledge into parameters inevitably causes lossy and conflicting compression. Identical structural patterns may carry opposite semantics across domains, and collapsing them into shared embeddings distorts meaning. Moreover, such compression is irreversible: once absorbed into weights, knowledge cannot be retrieved, verified, or updated without retraining, undermining transparency and grounded reasoning.

Challenge III: Entangled representation and storage. The parameter-based storage tightly entangles knowledge with representations, hindering efficient adaptation. Fine-tuning simultaneously adjusts task-specific features and updates memorized knowledge, making learning inefficient and data-intensive. This entanglement also obscures interpretability, as predictions cannot be traced to explicit evidence, reducing reliability in high-stakes applications.

Our key insight is to move beyond parameter-centric storage by externalizing graph knowledge, inspired by Retrieval-Augmented Generation (RAG) [22]. Unlike text, graph knowledge is fragmented across attributes and structures, making retrieval more challenging. Existing GFMs compress such evidence into parameters, losing explicit access and updatability. We argue that treating graph knowledge as first-class retrievable objects enables external storage, aligned pre-training, and scalable, interpretable adaptation.

In this work, we propose **RAG-GFM**, a **Retrieval-Augmented Generation aided Graph Foundation Model**. RAG-GFM incorporates three key components. First, we construct a dual-store unified retrieval database, consisting of a semantic store from prefix-structured text embeddings and a structural store from centrality-based motif encodings, enabling GFMs to query external knowledge on demand. Second, we design a dual-view alignment objective that contrasts semantic representation with structural subgraphs during pre-training, ensuring complementary representation learning

across modalities. Third, we introduce in-context sample augmentation, where retrieved texts and motifs are appended as contextual evidence for few-shot adaptation, enriching support instances with explicit external knowledge. **Our contributions are:**

- We propose RAG-GFM, the first retrieval-augmented graph foundation model that explicitly addresses in-memory bottlenecks.
- We design a dual-store retrieval module, a dual-view alignment objective, and an in-context sample augmentation mechanism, providing a unified pipeline for knowledge externalization, robust pre-training, and efficient adaptation.
- Extensive experiments on six benchmark graph datasets demonstrate that RAG-GFM consistently outperforms 13 state-of-the-art GFM baselines in both cross-domain node and graph classification, achieving superior effectiveness and efficiency.

2 Related Work

Graph Foundation Models (GFMs). GFMs extend large-scale pre-training to graphs via self-supervised learning [2, 4, 30, 47, 68, 71]. Most assume distributional similarity between pre-training and downstream tasks [16, 80], limiting robustness under domain shift. Recent work explores cross- or multi-domain learning, LLM alignment, domain tokens, and structural guarantees [14, 26, 26, 51–53, 61, 64, 70, 75, 78, 81], yet GFMs remain parameter-centric and struggle with structural and semantic consistency.

Retrieval-Augmented Generation (RAG). RAG enhances the LLMs by retrieving external knowledge to mitigate context limits and hallucinations [6]. Using lexical or semantic retrieval with query optimization and re-ranking [23, 32, 40], RAG achieves strong performance in QA and reasoning [17, 54]. Extensions to graph data motivate GraphRAG [13, 19, 33, 39, 49, 56, 66]. While GFM-RAG [31] uses GFMs to improve RAG, we instead leverage RAG to fundamentally enhance GFMs.

3 Notations and Preliminaries

Notations. We represent a graph as $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ denotes the set of nodes and $\mathcal{E}$ the set of edges. For a graph G_i sampled from any of the source domains, let $\mathbf{A} \in \{0, 1\}^{N_i \times N_i}$ be the adjacency matrix and $\mathbf{X} \in \mathbb{R}^{N_i \times d_i}$ be the node feature matrix. Here, $N_i = |\mathcal{V}_i|$ denotes the number of nodes, and d_i denotes the original input feature dimension. $\mathbf{Z}, \mathbf{W}, \mathbf{H}$ are the hidden representations.

Multi-domain Pre-training. Let $\mathcal{G}^S = \{G_1^S, \dots, G_n^S\}$ denote a collection of source graphs from domains $\mathcal{D}^S$, each associated with labels $\mathcal{Y}^S$. We cast pre-training as a self-supervised link prediction problem with universal templates [27], ensuring task consistency with downstream settings. The learner is defined as $h = g(f_\theta(\cdot))$, where the encoder $f: \mathbb{R}^{d_i} \mapsto \mathbb{R}^d$ produces node embeddings and the discriminator $g: \mathbb{R}^d \times \mathbb{R}^d \mapsto \mathbb{R}^2$ predicts link existence. Once the pre-training converges, parameter θ^* is frozen as the backbone.

Few-shot Fine-tuning. We consider graphs $\mathcal{G}^T$ from target domains $\mathcal{D}^T$ (seen or unseen). Under m -shot setting ($m \ll \sum_{i=1}^n N_i$), only m labeled samples $\mathcal{Y}^T$ are available. Fine-tuning applies the pre-trained $h = g(f_\theta^*(\cdot))$ augmented with learnable prompts $\mathcal{P}_\Omega$, where Ω denotes tunable parameters. Both node and graph classification (via node-centered ego-graphs) are reformulated as link prediction, maintaining homogeneity with pre-training.

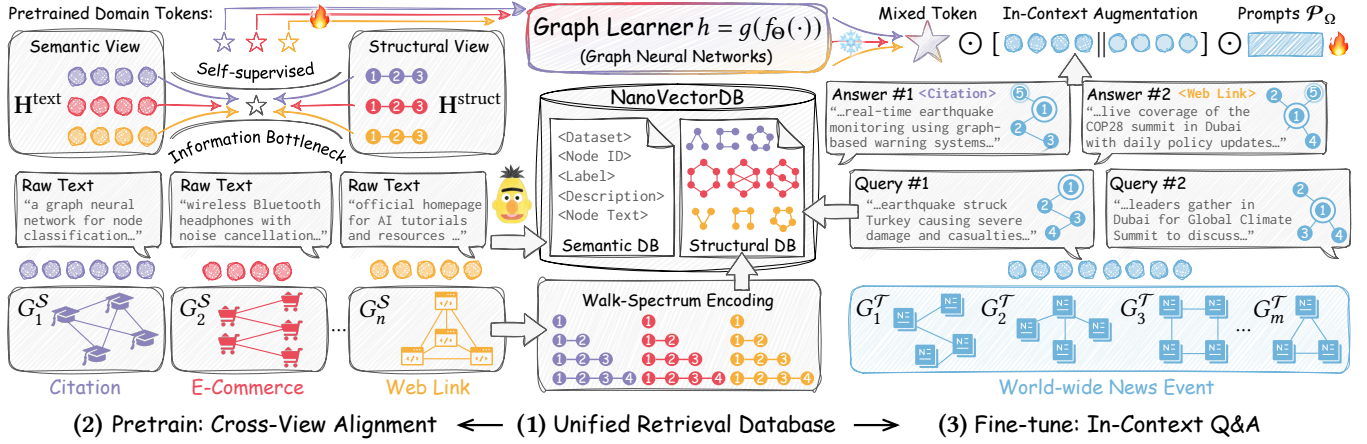


Figure 2: The framework of RAG-GFM. The framework includes three stages: (1) Unified Semantic-Structural Bi-Modal Retrieval Database for externalizing graph knowledge, (2) Cross-View Knowledge Alignment for pre-training transferable priors, and (3) In-Context Retrieval Augmentation for few-shot adaptation via domain-gated lightweight graph prompts.

4 Method

We illustrate the framework of RAG-GFM in Figure 2.

4.1 Unified Semantic-Structural Bi-Modal Retrieval Database

At the core of our framework lies a unified retrieval database that externalizes knowledge into two complementary modalities: a **semantic store** that organizes enriched node texts as retrievable documents, and a **structural store** that captures motif-level patterns. Formally, we denote the bi-modal database^{*} as $\mathcal{D} = \{\mathcal{D}_{\text{text}}, \mathcal{D}_{\text{struct}}\}$. Given query q and scoring function $s(\cdot, \cdot)$, the retrieval operator is:

$$\text{Retrieve}(\mathcal{D}, q, k) = \arg \text{Top-}k_{(u, z_u) \in \mathcal{D}} [s(q, z_u)], \quad (1)$$

with each entry (u, z_u) denoting a database record identified by u and described by z_u . $\mathcal{D}$ is queried in both pre-training and fine-tuning, which allows RAG-GFM to ground predictions in explicit semantic and structural evidence rather than obscure parameters.

4.1.1 Semantic Store. Unlike raw node features reduced to numerical vectors, most of the graphs are text-attributed, with nodes from sources such as abstracts, product descriptions, or profiles. We recover each node’s raw text t_v by tracing it back to its original corpus (e.g., metadata in citations) [24, 25], and treat it as a first-class signal. The semantic pipeline branches into two tracks:

On the **representation track**, we address the dimension-wise mismatch of raw features across domains using PCA [38]. For each graph G_i with feature matrix $X_i \in \mathbb{R}^{N_i \times d_i}$, we apply:

$$\tilde{X}_i^S = \text{PCA}_{d_0}(X_i^S) \in \mathbb{R}^{N_i \times d_0}. \quad (2)$$

In parallel, the raw text t_v is encoded by a BERT into a semantic vector $\mathbf{b}_v \in \mathbb{R}^{d_0}$. The updated node feature is $\hat{\mathbf{x}}_v^S = [\tilde{X}_v^S \parallel \mathbf{b}_v] \in \mathbb{R}^{2d_0}$, combining dimension-aligned attributes with enriched semantics. Thus, we learn the graph embeddings with the text-wise encoder:

$$\mathbf{Z}_i^S = f_{\theta_t}(\hat{X}_i^S, A_i^S) \in \mathbb{R}^{N_i \times d}. \quad (3)$$

^{*}Implemented in NanoVectorDB [69], a lightweight but efficient database that provides scalable storage and top- k retrieval over dense queries.

On the **retrieval track**, we build $\mathcal{D}_{\text{text}}$ as a textual vector database. For each node and its corresponding raw text, to standardize heterogeneous sources and make retrieval controllable and interpretable, we augment each document with a structured prefix:

Prefix Schema	Example
Dataset: <dataset_name>	<i>Cora</i>
Node ID: <node_id>	#123
Label: <node_label>	<i>Neural Networks</i>
Description: <description>	<i>Papers about neural networks.</i>
Node Text: <node_text>	<i>This paper introduces the LSTM, a Long Short-Term Memory model.</i>

The prefixed document $\tilde{t}_v$ is segmented not by naive length rules but into graph-aware chunks $\{c_{v_1}, \dots, c_{v_k}\}$ aligned with descriptive fields and class-level information for fine-grained retrieval. In this way, it yields coherent chunks that remain intrinsically aligned with the structure rather than arbitrary spans. Each chunk is embedded with BERT into $\tilde{z}_{v_j}^S \in \mathbb{R}^{768}$. We insert $(v, \tilde{z}_{v_j}^S, \text{meta}_{v_j})$ into the $\mathcal{D}_{\text{text}}$, where meta_{v_j} carries the structured fields from the prefix. Formally,

$$\mathcal{D}_{\text{text}} = \{(v, \tilde{z}_{v_j}^S, \text{meta}_{v_j}) \mid v \in \{\mathcal{V}_i\}_{i=1}^n, j \in [1, k]\}. \quad (4)$$

The prefix serves as a “retrieval hook” for the metadata filtering and cross-domain alignment, while the 768-dimensional embeddings preserve semantic capacity for in-context augmentation.

4.1.2 Structural Store. Enumerating motifs is computationally intractable (NP-hard), and storing arbitrary subgraphs introduces noise. Inspired by [5, 45], we propose the **Walk-Spectrum Encoding (WSE)**, which ranks nodes by a walk-based importance and encodes their local neighborhoods with a multi-order walk signature.

Definition 1 (Walk-Spectrum Encoding). For a node $v \in \mathcal{V}$, the Walk-Spectrum Encoding (WSE) of order K is defined as:

$$C_\alpha^{\text{WSE}}(v) = [\alpha A_{vv}, \alpha^2 A_{vv}^2, \alpha^3 A_{vv}^3, \dots, \alpha^K A_{vv}^K], \quad (5)$$

where $\alpha \in (0, 1)$ is a damped variant, and A_{vv}^k counts the number of closed walks of length k starting and ending at node v .

WSE summarizes a node's participation in closed walks of varying lengths, thereby encoding structural patterns beyond any fixed-radius neighbors. This motivates the following result on its ability to separate graphs that local methods [45] cannot distinguish:

Proposition 1 (Structural Separability of WSE). There exist pairs of non-isomorphic graphs G_1, G_2 and nodes $v \in G_1, u \in G_2$ such that for any fixed radius r , the r -hop neighbors $\mathcal{N}_r(v)$ and $\mathcal{N}_r(u)$ are isomorphic, yet the Walk-Spectrum Encodings satisfy:

$$\mathbf{C}_\alpha^{\text{WSE}}(v) \neq \mathbf{C}_\alpha^{\text{WSE}}(u). \quad (6)$$

Proofs in Appendix B.1. While WSE provides rich structural signatures, computing and storing subgraphs for all nodes is infeasible at scale. To address this, we derive an anchor scoring function:

$$r_v(\alpha, K) = \sum_{k=1}^K \alpha^k \mathbf{A}_{vv}^k, \quad \text{for each } v \in \{\mathcal{V}_i\}_{i=1}^n \quad (7)$$

Intuitively, r_v highlights nodes most recurrently involved in structural motifs. Ranking nodes by r_v allows us to select a compact yet informative pool of anchors for motif storage and retrieval.

We then select the top- M nodes in each graph into $\mathcal{V}_{\text{anchor}}^S$, and extract its h -hop ego-subgraph $G_{v(h)}^S$ for each node v . To reduce redundancy, overlapping ego-subgraphs are pruned via node set equivalence. The resulting structural store is defined as:

$$\mathcal{D}_{\text{struct}} = \{(v, G_{v(h)}^S, \mathbf{C}_\alpha^{\text{WSE}}(v), \text{meta}_v) \mid v \in \mathcal{V}_{\text{anchor}}^S\}, \quad (8)$$

where meta_v includes metadata like hop radius, anchor score, *etc.* At this point, we have established the unified semantic-structural bi-modal retrieval database $\mathcal{D}$, which will serve as the foundation for subsequent pre-training and fine-tuning.

4.2 Cross-View Knowledge Alignment for Multi-domain Pre-training

With the unified database $\mathcal{D} = \{\mathcal{D}_{\text{text}}, \mathcal{D}_{\text{struct}}\}$ in place, the next step is to pre-train the encoder $f_\Theta(\cdot)$ that couples semantic and structural information in a principled way. The goal is not to collapse them into a single representation but to ensure that both carry complementary and transferable signals across domains.

4.2.1 Node Views. For each G_i^S , we build two node-level views. The semantic view is given by the enriched node embeddings $\mathbf{Z}_i^S$ in Eq. (3), which combine raw attributes and text-derived features. The structural view is constructed from the walk-spectrum encoding:

$$\mathbf{W}_i^S = [\mathbf{C}_\alpha^{\text{WSE}}(v)]_{v \in \mathcal{V}_i^S}, \quad (9)$$

where each item records closed-walk signatures up to order K , capturing recurring motif patterns and higher-order relational signals.

Domain Tokens. To incorporate domain-level priors, we introduce a learnable token $\tau_{D_i} \in \mathbb{R}^{d_\tau}$ for each source domain D_i^S , which is concatenated to every node representation before encoding:

$$\bar{\mathbf{Z}}_i^S = [\mathbf{Z}_i^S \parallel \mathbf{1} \cdot \tau_{D_i}^\top], \quad \bar{\mathbf{W}}_i^S = [\mathbf{W}_i^S \parallel \mathbf{1} \cdot \tau_{D_i}^\top], \quad (10)$$

where $\mathbf{1}$ denotes a broadcast vector ensuring nodes within a domain share this token. During optimization, τ_{D_i} accumulates domain priors that are not captured by individual nodes, such as global semantics in citation graphs or biochemical motifs in protein-protein networks. Tokens initialize lightweight graph prompts at fine-tuning, enabling adaptation without revisiting the full pre-training corpus.

4.2.2 Cross-View Information Bottleneck. Our pre-training is entirely self-supervised: the key idea is to align semantic and structural views of the same node without relying on labels, while simultaneously preventing collapse by encouraging each view to preserve modality-specific information. We apply two encoders over the same topology but different features:

$$\mathbf{H}_i^{\text{text}} = f_{\Theta_t}(\bar{\mathbf{Z}}_i^S, \mathbf{A}_i^S), \quad \mathbf{H}_i^{\text{struct}} = f_{\Theta_s}(\bar{\mathbf{W}}_i^S, \mathbf{A}_i^S), \quad (11)$$

which yields semantic embeddings $\mathbf{h}_{i,v}^{\text{text}}$ and structural embeddings $\mathbf{h}_{i,v}^{\text{struct}}$ for each node v . Concretely, we introduce the self-supervised information bottleneck [58] by maximizing the mutual information between semantic and structural embeddings, and applying compression regularizers to discard redundant signals:

$$\mathcal{L}_{\text{align}}^{(i,v)} = \underbrace{-I(\mathbf{h}_{i,v}^{\text{text}}; \mathbf{h}_{i,v}^{\text{struct}})}_{\text{relevance}} + \underbrace{\beta \left(I(\mathbf{h}_{i,v}^{\text{text}}; \bar{\mathbf{z}}_i^S) + I(\mathbf{h}_{i,v}^{\text{struct}}; \bar{\mathbf{w}}_i^S) \right)}_{\text{compression}}, \quad (12)$$

where $I(\cdot; \cdot)$ denotes the mutual information, which is intractable over unknown latent distributions of variables, and β is the trade-off hyper-parameter. We adopt a contrastive approximation that yields a variational bound for tractable computation [21, 48, 57, 58]:

Proposition 2 (Cross-View Mutual Information Bounds).

The relevance term admits the InfoNCE lower-bound estimator:

$$I(\mathbf{h}_{i,v}^{\text{text}}; \mathbf{h}_{i,v}^{\text{struct}}) \geq \frac{1}{|\mathcal{B}|} \sum_{v \in \mathcal{B}} \log \frac{\exp(\sigma(g_t(\mathbf{h}_{i,v}^{\text{text}}), g_s(\mathbf{h}_{i,v}^{\text{struct}}))/\tau)}{\sum_{u \in \mathcal{B}} \exp(\sigma(g_t(\mathbf{h}_{i,v}^{\text{text}}), g_s(\mathbf{h}_{i,u}^{\text{struct}}))/\tau)}, \quad (13)$$

where g_t, g_s are projections, $\sigma(\cdot)$ is similarity, τ is a temperature, positives are formed by the same node across the views (v, v) in a batch $\mathcal{B}$, and negatives by mismatched nodes $(v, u), u \neq v$.

The compression term can be upper-bounded via KL-divergence:

$$I(\mathbf{h}_{i,v}^{\text{text}}; \bar{\mathbf{x}}_i^S) \leq \mathbb{E}_{p(\mathbf{h}_{i,v}^{\text{text}} | \bar{\mathbf{x}}_i^S)} [\log q_\phi(\mathbf{h}_{i,v}^{\text{text}} | \bar{\mathbf{x}}_i^S)] - \mathbb{E}_{p(\mathbf{h}_{i,v}^{\text{text}})} [\log p(\mathbf{h}_{i,v}^{\text{text}})], \quad (14)$$

where v is sampled from $\mathcal{B}$, $\mathbf{x}$ denotes $\mathbf{z}$ or $\mathbf{w}$, and $q_\phi(\cdot | \cdot)$ is a variational approximation of the true conditional distribution.

Proposition 2 provides tractable self-supervised estimators for the otherwise intractable mutual information terms, with lower bounds applied to cross-view alignment and upper bounds applied to view-specific compression. We provide sketch proofs in Appendix B.2.

4.2.3 Pre-training Objective. Bringing the above components together, the overall pre-training objective is defined as:

$$\mathcal{L}_{\text{pretrain}}(\Theta_t, \Theta_s) = \sum_{D_i^S} \frac{1}{|\mathcal{V}_i^S|} \sum_{v \in \mathcal{V}_i^S} \mathcal{L}_{\text{align}}^{(i,v)} \cdot \gamma \sum_{D_i^S} \|\tau_{D_i}\|_2^2, \quad (15)$$

where the first term aggregates the cross-view alignment loss across source domains, and the second term regularizes domain tokens to prevent overfitting. γ acts as their trade-off hyper-parameter.

In practice, mini-batches are constructed by mixing nodes from different domains, and the corresponding domain tokens are updated jointly with semantic and structural encoders. This setup enforces cross-domain consistency during pre-training while preserving domain-specific priors for downstream adaptation. Θ_t and Θ_s are frozen once pre-training converges. We illustrate the pre-training pipeline in Algorithm 1 with its complexity analysis.

4.3 In-Context Retrieval Augmentation for Few-Shot Fine-Tuning

We proceed to fine-tune pre-trained model under *meta-learning* settings (m -shot), which is more challenging in real-world scenarios.

4.3.1 Domain-Gated Fusion. To ensure dimension-consistency with pre-training, each support sample is processed by the same representation track (Section 4.1.1, Eq. (2) and Eq. (3)) into $\hat{\mathbf{X}}^\mathcal{T}$.

Before retrieval, we estimate domain affinities that will route external evidence. For each target node v_i or graph $G_i^\mathcal{T}$, we compute soft gating weights over source domains via domain tokens:

$$\pi_{i,k} = \frac{\exp(\sigma(\mathbf{Z}_i^\mathcal{T}, \tau_{D_k}))}{\sum_j \exp(\sigma(\mathbf{Z}_i^\mathcal{T}, \tau_{D_j}))}, \quad \mathbf{Z}_i^\mathcal{T} = f_{\Theta_t}^*(\hat{\mathbf{X}}_i^\mathcal{T}, \mathbf{A}_i^\mathcal{T}), \quad (16)$$

where $\mathbf{Z}_i^\mathcal{T}$ is the pre-trained encoder output on $\hat{\mathbf{X}}^\mathcal{T}$. These $\{\pi_{i,k}\}_{k=1}^n$ act as domain-aware gates reused by the later augmentations.

4.3.2 Query and Retrieval. For clarity, we present query and retrieval in node-level settings. The extension to graph-level tasks follows directly by treating each graph as a single instance.

Semantic Retrieval. For each few-shot target node v and its one-hop neighbors, we form a *textual query* $\mathbf{q}_v^{\text{text}}$ from the raw text and submit it to $\mathcal{D}_{\text{text}}$, restricting search to pre-training domains to avoid leakage. The database returns top- k *textual answers* $\{\tilde{\mathbf{z}}_u^S\}$, which are aggregated through softmax-weighted fusion:

$$(\Delta \mathbf{z}_v^\mathcal{T})^{\text{text}} = \sum_{u \in \text{Top-}k(v)} w_{vu} \cdot \tilde{\mathbf{z}}_u^S, \quad w_{vu} = \frac{\exp(\sigma(\mathbf{q}_v^{\text{text}}, \tilde{\mathbf{z}}_u^S))}{\sum_{u'} \exp(\sigma(\mathbf{q}_v^{\text{text}}, \tilde{\mathbf{z}}_{u'}^S))}, \quad (17)$$

where $\mathbf{z}_v^\mathcal{T}$ is in-context augmented with hyper-parameter λ_{text} :

$$\mathbf{z}_v^{\mathcal{T}'} = \mathbf{z}_v^\mathcal{T} + \lambda_{\text{text}} \cdot (\Delta \mathbf{z}_v^\mathcal{T})^{\text{text}}. \quad (18)$$

Structural Retrieval. For the same node v and its neighbors, we extract the h -hop subgraph, encode it with WSE as the *structural query*, and submit $\mathbf{q}_v^{\text{struct}}$ to $\mathcal{D}_{\text{struct}}$. From each source domain D_i^S , we retrieve the most structurally similar motif $G_{v(h),i}^S = (\hat{\mathbf{X}}_{v(h),i}^\mathcal{T}, \mathbf{A}_{v(h),i}^\mathcal{T})$ as *structural answer*. We then fuse cross-domain answers using domain gates $\{\pi_{v,k}\}$ with hyper-parameter λ_{struct} :

$$\mathbf{z}_v^{\mathcal{T}''} = \mathbf{z}_v^{\mathcal{T}'} + \lambda_{\text{struct}} \cdot (\Delta \mathbf{z}_v^\mathcal{T})^{\text{struct}}, \quad (19)$$

$$(\Delta \mathbf{z}_v^\mathcal{T})^{\text{struct}} = \sum_{D_i^S} \pi_{v,k} (f_{\Theta_s}^*(\hat{\mathbf{X}}_{v(h),i}^\mathcal{T}, \mathbf{A}_{v(h),i}^\mathcal{T})). \quad (20)$$

4.3.3 Prompted Few-shot Adaptation. Given m retrieved and augmented support samples $\{(\mathbf{h}_i^\mathcal{T}, \mathbf{y}_i)\}$, to enable efficient adaptation without updating the frozen Θ_t and Θ_s , we initialize learnable graph prompts $\mathcal{P}_\Omega$ by the routed domain priors:

$$\mathbf{h}_i^\mathcal{T} = [\mathbf{z}_i^{\mathcal{T}''} \| \mathcal{P}_\Omega], \quad \mathcal{P}_\Omega \leftarrow \sum_{k=1}^n \pi_{i,k} \tau_{D_k}, \quad (21)$$

where $\mathbf{h}_i^\mathcal{T}$ denotes the i -th target node or graph embedding. The fine-tuning objective is transformed into determining the similarity between the query sample and the class prototype embedding:

$$\mathcal{L}_{\text{fine-tune}}(\mathcal{P}_\Omega) = - \sum_{\{(\mathbf{h}_i^\mathcal{T}, \mathbf{y}_i)\}} \left[\log \frac{\exp(g(\mathbf{h}_i^\mathcal{T}, \bar{\mathbf{h}}_{\mathbf{y}_i}^\mathcal{T})/\tau)}{\sum_{\mathbf{y}_j \in \{\mathcal{Y}^\mathcal{T}\}} \exp(g(\mathbf{h}_i^\mathcal{T}, \bar{\mathbf{h}}_{\mathbf{y}_j}^\mathcal{T})/\tau)} \right], \quad (22)$$

where $\bar{\mathbf{h}}_{\mathbf{y}_i}^\mathcal{T}$ is the class prototype for samples in class $\mathbf{y}_i$. We analyse the fine-tuning pipeline in Algorithm 2 with complexity analysis.

4.4 Algorithms and Complexity

Shown in Appendix A, RAG-GFM consists of two stages. In pre-training, dual-view encoding and self-supervised alignment dominate the cost, yielding $O(L(E_\mathcal{B} + |\mathcal{B}|)d + |\mathcal{B}|^2d)$ per iteration, where $E_\mathcal{B}$ is the edge count in batch $\mathcal{B}$. In fine-tuning, semantic retrieval and structural motif retrieval are combined via domain-gated fusion and prompt-based classification, giving $O(m[\log M_{\text{text}} + n \log M_{\text{struct}} + (k + n + C)d])$ per iteration, with M_{text} and M_{struct} the database sizes and C the class number. Retrieval adds only logarithmic overhead, while adaptation updates prompts instead of full parameters, ensuring much lower cost than end-to-end fine-tuning.

Overall, the complexity remains comparable to state-of-the-art GFMs while achieving superior efficiency in few-shot adaptation.

5 Experiment

We evaluate RAG-GFM[†], focusing on the these research questions:

- **RQ1:** How effective on cross-dataset or cross-domain few-shot node and graph classification? ($\triangleright$ Section 5.2)
- **RQ2:** Which module contributes most? ($\triangleright$ Section 5.3)
- **RQ3:** Can LLM achieve zero-shot reasoning? ($\triangleright$ Section 5.4)
- **RQ4:** How efficient in time and memory? ($\triangleright$ Section 5.5)
- **RQ5:** How reliable and interpretable is RAG? ($\triangleright$ Section 5.6)
- **RQ6:** How sensitive to hyper-parameter changes? ($\triangleright$ Section 5.7)

5.1 Experimental Settings

5.1.1 Datasets. To emphasize the pre-training capability across heterogeneous domains, we adopt *five* benchmark text-attributed graph datasets spanning *three* distinct domains. This design contrasts with conventional settings that often regard a single dataset as an independent domain, offering a more challenging evaluation.

- **Citation Domain:** Cora [35], CiteSeer [8], PubMed [42].
- **E-Commerce Domain:** Ogbn-Products [15] from a large-scale product co-purchase network, which includes sub-categories.
- **Web Link Domain:** Wiki-CS [36], a hyperlink web page network constructed from a subset of Wikipedia.

5.1.2 Baselines. We compare RAG-GFM with *13* state-of-the-art baselines from *four* primary categories.

- **Vanilla GNNs:** GCN [20] and GAT [59] without pre-training.
- **Graph Pre-training:** DGI [60], InfoGraph [46], GraphCL [72].
- **Text-free GFMs:** GCOPE [81], MDGPT [75], SAMGPT [74], and MDGFM [64], which are evaluated on text-free graphs.
- **Text-attributed GFMs:** OFA [25], ZeroG [24], GraphCLIP [83], and UniGraph [14], which are evaluated on text-attributed graphs.

5.1.3 Pre-training and Fine-tuning Settings. We evaluate node- and graph-level classification under the m -shot setting, where m labeled samples per class are randomly selected. For graph task, ego-graphs centered on target nodes are extracted and labeled by central nodes [28, 73, 75]. To assess generalization, we adopt two leave-out strategies, both referred to as **LODO**: (1) **Leave-One-Dataset-Out**, holding out one dataset as target; and (2) **Leave-One-Domain-Out**, excluding an entire domain during pre-training. These variants capture transferability across unseen datasets and unseen domains. Results are reported by mean values with standard deviation.

[†] <https://github.com/RingBDStack/RAG-GFM>.

Table 1: Few-shot classification results under the LODO setting. We report mean accuracy (%) with standard deviation. “LODO (dataset)” denotes training on all datasets except the target, irrespective of domain. “LODO (domain)” denotes training with all datasets excluding those belonging to the target domain. Best results are presented in bold and the runner-ups are underlined.

Setting	LODO (dataset)						LODO (domain)			
Target Dataset	Cora		CiteSeer		PubMed		Ogbn-Products		Wiki-CS	
<i>m</i> -shot	1	5	1	5	1	5	1	5	1	5
Method	Node Classification									
GCN ($\triangleright$ ICLR'17)	28.4 $\pm$ 4.6	50.2 $\pm$ 4.9	29.3 $\pm$ 3.4	45.9 $\pm$ 5.4	40.3 $\pm$ 6.9	50.7 $\pm$ 7.5	44.7 $\pm$ 4.3	48.1 $\pm$ 3.4	37.2 $\pm$ 5.1	48.1 $\pm$ 4.9
GAT ($\triangleright$ ICLR'18)	29.7 $\pm$ 5.2	49.0 $\pm$ 7.9	29.3 $\pm$ 3.5	46.1 $\pm$ 5.1	40.5 $\pm$ 4.0	52.2 $\pm$ 6.3	44.6 $\pm$ 4.0	48.1 $\pm$ 4.5	37.9 $\pm$ 4.5	48.6 $\pm$ 4.5
DCI ($\triangleright$ ICLR'19)	30.8 $\pm$ 3.9	49.9 $\pm$ 6.6	31.4 $\pm$ 4.1	46.5 $\pm$ 7.1	40.0 $\pm$ 5.9	53.6 $\pm$ 7.1	46.0 $\pm$ 5.4	50.1 $\pm$ 4.2	38.1 $\pm$ 5.1	49.2 $\pm$ 4.4
GraphCL ($\triangleright$ NeurIPS'20)	33.6 $\pm$ 5.8	53.2 $\pm$ 5.4	28.2 $\pm$ 3.1	48.8 $\pm$ 7.7	39.0 $\pm$ 8.7	54.7 $\pm$ 4.4	46.1 $\pm$ 5.0	50.5 $\pm$ 4.6	40.0 $\pm$ 4.0	50.1 $\pm$ 5.2
GCOPE ($\triangleright$ KDD'24)	36.3 $\pm$ 3.9	55.6 $\pm$ 6.4	40.4 $\pm$ 4.6	56.9 $\pm$ 5.8	44.8 $\pm$ 4.7	53.6 $\pm$ 8.6	47.7 $\pm$ 4.9	51.4 $\pm$ 3.3	45.8 $\pm$ 5.5	53.5 $\pm$ 4.7
MDGPT ($\triangleright$ arXiv'24)	42.6 $\pm$ 6.8	62.7 $\pm$ 6.0	37.9 $\pm$ 7.2	55.9 $\pm$ 3.3	51.0 $\pm$ 9.0	58.7 $\pm$ 6.2	49.1 $\pm$ 6.0	56.6 $\pm$ 2.7	45.0 $\pm$ 4.8	54.1 $\pm$ 5.2
SAMGPT ($\triangleright$ WWW'25)	46.8 $\pm$ 6.5	64.6 $\pm$ 6.7	38.7 $\pm$ 6.4	56.4 $\pm$ 4.7	51.9 $\pm$ 9.5	59.1 $\pm$ 6.0	49.8 $\pm$ 4.4	56.2 $\pm$ 3.3	44.4 $\pm$ 5.5	54.4 $\pm$ 5.8
MDGFM ($\triangleright$ ICML'25)	47.4 $\pm$ 6.3	66.0 $\pm$ 6.5	36.3 $\pm$ 6.2	55.8 $\pm$ 4.0	50.2 $\pm$ 8.8	58.4 $\pm$ 6.4	48.5 $\pm$ 4.7	54.7 $\pm$ 4.9	43.4 $\pm$ 5.8	53.9 $\pm$ 4.4
OFA ($\triangleright$ ICLR'24)	45.9 $\pm$ 6.3	67.7 $\pm$ 2.9	38.0 $\pm$ 7.6	52.8 $\pm$ 6.4	46.3 $\pm$ 6.0	56.0 $\pm$ 5.9	49.1 $\pm$ 5.7	55.3 $\pm$ 4.2	42.8 $\pm$ 4.6	54.3 $\pm$ 4.0
ZeroG ($\triangleright$ KDD'24)	51.8 $\pm$ 5.6	71.4 $\pm$ 1.7	39.7 $\pm$ 5.9	54.6 $\pm$ 2.0	53.1 $\pm$ 3.5	63.0 $\pm$ 3.5	53.2 $\pm$ 2.9	59.9 $\pm$ 3.1	<u>46.1 $\pm$ 3.4</u>	59.0 $\pm$ 2.0
GraphCLIP ($\triangleright$ WWW'25)	53.9 $\pm$ 5.3	73.1 $\pm$ 2.9	<u>40.6 $\pm$ 3.4</u>	55.2 $\pm$ 1.2	56.8 $\pm$ 1.9	65.2 $\pm$ 2.8	53.4 $\pm$ 6.1	<u>62.6 $\pm$ 4.3</u>	45.5 $\pm$ 2.1	59.9 $\pm$ 2.8
UniGraph ($\triangleright$ KDD'25)	<u>56.1 $\pm$ 6.3</u>	<u>74.8 $\pm$ 1.9</u>	40.5 $\pm$ 3.1	<u>56.3 $\pm$ 2.2</u>	<u>57.0 $\pm$ 3.1</u>	<u>66.8 $\pm$ 2.4</u>	<u>53.8 $\pm$ 3.4</u>	61.0 $\pm$ 3.5	45.1 $\pm$ 3.6	58.4 $\pm$ 3.1
RAG-GFM (ours)	58.4 $\pm$ 6.0	76.1 $\pm$ 0.7	41.5 $\pm$ 3.0	57.7 $\pm$ 1.8	59.2 $\pm$ 2.7	68.7 $\pm$ 1.8	55.4 $\pm$ 7.6	64.2 $\pm$ 4.4	47.8 $\pm$ 3.8	60.9 $\pm$ 2.5
Method	Graph Classification									
GCN ($\triangleright$ ICLR'17)	40.1 $\pm$ 4.8	52.9 $\pm$ 4.1	29.5 $\pm$ 5.7	43.9 $\pm$ 5.9	45.3 $\pm$ 7.3	55.4 $\pm$ 5.3	47.6 $\pm$ 3.2	52.6 $\pm$ 5.3	38.9 $\pm$ 4.1	41.5 $\pm$ 6.4
GAT ($\triangleright$ ICLR'18)	36.0 $\pm$ 5.1	49.6 $\pm$ 5.1	26.0 $\pm$ 7.8	45.3 $\pm$ 7.3	41.0 $\pm$ 5.8	54.5 $\pm$ 7.3	49.2 $\pm$ 5.6	52.9 $\pm$ 5.5	38.3 $\pm$ 4.6	41.1 $\pm$ 3.5
InfoGraph ($\triangleright$ ICLR'20)	42.2 $\pm$ 5.2	54.7 $\pm$ 4.9	30.2 $\pm$ 4.1	47.2 $\pm$ 5.2	49.1 $\pm$ 5.4	59.7 $\pm$ 7.1	50.7 $\pm$ 4.3	53.8 $\pm$ 5.1	40.4 $\pm$ 4.3	42.4 $\pm$ 5.0
GraphCL ($\triangleright$ NeurIPS'20)	39.6 $\pm$ 5.8	55.2 $\pm$ 5.9	32.6 $\pm$ 6.5	46.4 $\pm$ 3.8	47.7 $\pm$ 7.0	60.0 $\pm$ 5.4	51.7 $\pm$ 5.9	53.0 $\pm$ 5.2	40.8 $\pm$ 4.6	42.5 $\pm$ 4.3
GCOPE ($\triangleright$ KDD'24)	55.9 $\pm$ 7.4	63.9 $\pm$ 4.8	41.0 $\pm$ 9.0	58.2 $\pm$ 5.8	54.4 $\pm$ 8.6	66.4 $\pm$ 3.7	55.8 $\pm$ 4.3	57.7 $\pm$ 4.8	42.2 $\pm$ 5.8	49.8 $\pm$ 3.5
MDGPT ($\triangleright$ arXiv'24)	52.8 $\pm$ 6.7	65.1 $\pm$ 4.2	41.0 $\pm$ 9.7	59.3 $\pm$ 6.0	55.5 $\pm$ 8.3	67.6 $\pm$ 4.6	54.5 $\pm$ 4.8	60.5 $\pm$ 3.4	43.2 $\pm$ 6.2	48.9 $\pm$ 4.2
SAMGPT ($\triangleright$ WWW'25)	53.3 $\pm$ 4.3	69.3 $\pm$ 3.4	42.4 $\pm$ 7.3	62.4 $\pm$ 5.7	57.7 $\pm$ 6.3	68.0 $\pm$ 4.6	54.4 $\pm$ 3.2	60.8 $\pm$ 4.8	43.5 $\pm$ 5.6	48.3 $\pm$ 5.7
MDGFM ($\triangleright$ ICML'25)	55.5 $\pm$ 5.4	69.4 $\pm$ 2.1	43.4 $\pm$ 6.4	60.8 $\pm$ 5.1	56.0 $\pm$ 5.1	67.1 $\pm$ 5.1	54.7 $\pm$ 2.1	59.8 $\pm$ 5.3	41.8 $\pm$ 6.7	46.4 $\pm$ 3.2
OFA ($\triangleright$ ICLR'24)	58.0 $\pm$ 3.7	65.1 $\pm$ 3.9	45.4 $\pm$ 6.6	60.0 $\pm$ 6.6	59.7 $\pm$ 4.3	67.2 $\pm$ 3.4	56.0 $\pm$ 3.9	60.1 $\pm$ 3.5	42.4 $\pm$ 5.8	48.1 $\pm$ 2.2
ZeroG ($\triangleright$ KDD'24)	65.1 $\pm$ 2.2	74.2 $\pm$ 1.6	50.3 $\pm$ 5.7	<u>64.0 $\pm$ 5.1</u>	61.4 $\pm$ 4.0	70.2 $\pm$ 1.3	58.5 $\pm$ 4.0	<u>66.2 $\pm$ 3.9</u>	46.1 $\pm$ 5.9	57.8 $\pm$ 3.6
GraphCLIP ($\triangleright$ WWW'25)	65.9 $\pm$ 3.7	75.1 $\pm$ 2.2	50.4 $\pm$ 4.0	63.0 $\pm$ 3.3	60.7 $\pm$ 3.8	71.3 $\pm$ 2.2	<u>58.6 $\pm$ 2.2</u>	65.7 $\pm$ 2.2	46.0 $\pm$ 3.3	58.8 $\pm$ 4.4
UniGraph ($\triangleright$ KDD'25)	<u>66.5 $\pm$ 2.5</u>	<u>76.5 $\pm$ 1.0</u>	50.9 $\pm$ 4.4	64.0 $\pm$ 2.4	<u>61.5 $\pm$ 2.6</u>	<u>71.4 $\pm$ 2.3</u>	58.1 $\pm$ 4.0	66.0 $\pm$ 3.8	<u>47.0 $\pm$ 2.5</u>	<u>58.9 $\pm$ 2.2</u>
RAG-GFM (ours)	68.7 $\pm$ 1.5	78.4 $\pm$ 0.6	52.2 $\pm$ 6.1	65.5 $\pm$ 2.2	62.4 $\pm$ 2.1	71.5 $\pm$ 1.9	60.2 $\pm$ 4.2	68.0 $\pm$ 3.1	48.1 $\pm$ 1.1	62.0 $\pm$ 4.3

5.2 RQ1: Transfer across Domains and Tasks

Table 1 reports the results of few-shot node and graph classification under both LODO settings. Results reveal that:

(1) **Overall superiority.** The proposed RAG-GFM consistently outperforms all baselines over each target graph. The advantage is most evident in the challenging LODO (domain) case, where it raises the 5-shot graph classification accuracy on Wiki-CS by over 5.3% compared with UniGraph, relatively. Baselines generally struggle as they compress knowledge entirely into parameters or rely only on texts, limiting generalization to unseen domains.

(2) **Retrieval enhances transfer.** In LODO (dataset) setting, RAG-GFM consistently outperforms parameter-only GFM, with the average relative gains of $\sim 3.0\%$. While baselines can still leverage shared domain priors, their parameter-centric representations fail to capture sufficient diversity across datasets. By contrast, retrieval from the unified database introduces complementary evidence: semantic queries supply textual signals, and structural queries provide transferable motifs, enabling adaptation with minimal supervision.

(3) **Cross-view alignment strengthens cross-domain robustness.** In the stricter LODO (domain) setting, where the entire target domain is unseen, the performance gap widens further with an average relative improvement of $\sim 4\%$. Baselines relying on text-only or domain-specific features degrade sharply, since they cannot bridge modality and domain gaps. In contrast, cross-view alignment in RAG-GFM enforces consistency between semantic and structural views, reducing overfitting to pre-training domains and ensuring that retrieved knowledge remains useful.

(4) **Domain-gated prompting ensures universality.** Consistent gains across tasks (on average 4.5% higher accuracy in the node task and 3.8% in the graph task, relatively) demonstrate that the framework is not tailored to a specific scenario. Baselines often overfit to one task formulation: models tuned for node classification transfer less effectively to graph classification. By introducing domain-gated prompts, our RAG-GFM adapts flexibly to both granularities, which is particularly advantageous in few-shot scenarios where labeled data is extremely scarce.

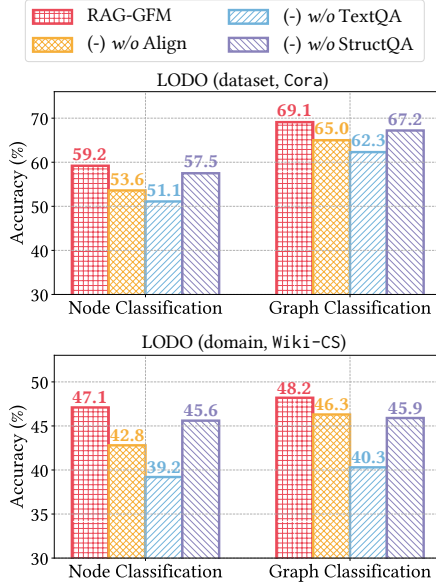


Figure 3: Ablation Study.

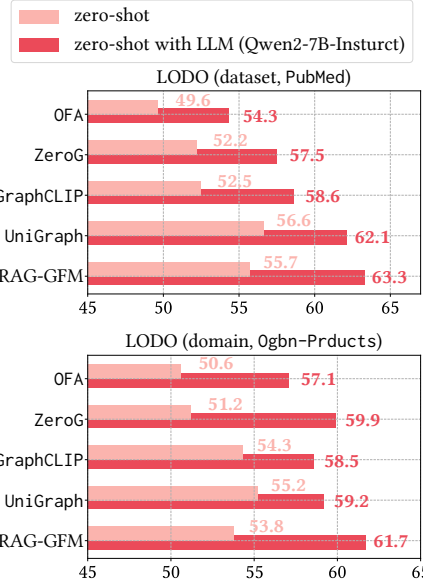


Figure 4: Zero-shot Reasoning.

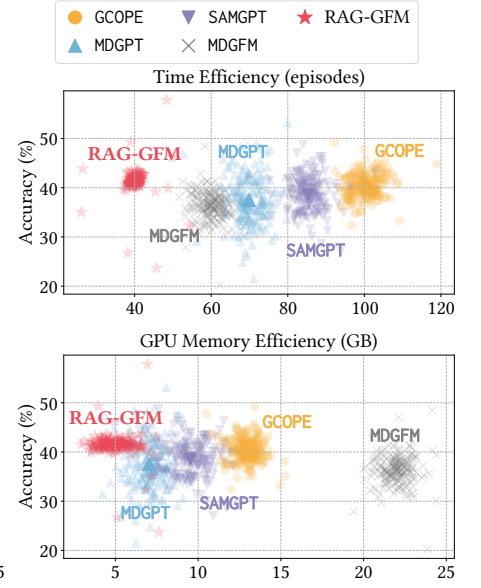


Figure 5: Efficiency Analysis on CiteSeer.

5.3 RQ2: Ablation Study

We conduct ablation studies on *three* core modules:

- **RAG-GFM (w/o Align)**: remove the cross-view knowledge alignment in pre-training (Section 4.2). Semantic and structural encoders are trained independently without mutual consistency.
- **RAG-GFM (w/o TextQA)**: remove the semantic retrieval in fine-tuning (Section 4.3). Textual augmentation is disabled and relies only on structural retrieval and parameterized features.
- **RAG-GFM (w/o StructQA)**: remove the structural retrieval in fine-tuning (Section 4.3). Structural augmentation is discarded, leaving only textual retrieval and parameterized features.

Results in Figure 3 demonstrate the full RAG-GFM achieves the best results across both settings. RAG-GFM (w/o Align) causes clear drops (e.g., 59.2% to 53.6% on Cora), underscoring the importance of semantic-structural consistency in pre-training. RAG-GFM (w/o TextQA) leads to the largest decline (nearly 8% on Wiki-CS), showing that raw attributes alone are insufficient and external semantic evidence is essential. RAG-GFM (w/o StructQA) also reduces accuracy (e.g., 69.1% to 67.2% on Cora), though less severely, indicating that motif-level cues provide secondary but stable benefits.

5.4 RQ3: Zero-shot Reasoning with LLMs

To in-depth examine the potential of large language models (LLMs), we evaluate a **zero-shot** setting without fine-tuning. Two scenarios are compared: (1) **zero-shot**, where the pre-trained models directly predict without supervision, and (2) **zero-shot with LLM**, where the graph task is reformulated into language queries (e.g., “Which class does this node belong to?”). Each target node is augmented with retrieved textual and structural context from the pre-training database, concatenated with its raw description, and fed into an LLM (we use Qwen2-7B-Instruct [55]) to produce predictions. This setup allows us to assess whether external language priors can compensate for the absence of labeled examples.

As baselines, we select GFM for text-attributed graphs, most of which already leverage LLMs as feature enhancers or semantic aligners during pre-training. However, these designs do not directly test whether LLMs themselves can serve as zero-shot reasoners.

Results in Figure 4 demonstrate that while RAG-GFM is competitive as an LLM-free GFM, it is sometimes slightly behind LLM-enhanced baselines in the zero-shot case. Notably, once equipped with LLM reasoning, it consistently achieves the best performance, improving from 55.7% to 63.3% on PubMed and from 53.8% to 61.7% on Ogbn-Products, surpassing all baselines. More importantly, the gains are not simply due to invoking stronger LLMs: by grounding reasoning in our unified dual-modal retrieval database, the prompts provide structured, domain-aligned evidence that enables LLMs to generalize more faithfully across unseen graphs. Furthermore, even existing LLM-enhanced GFMs benefit from our retrieval-augmented prompting, highlighting that RAG-GFM is not only effective in its own design but also serves as a general, pluggable enhancement that can universally elevate zero-shot reasoning in graph learning.

5.5 RQ4: Time and Memory Efficiency

We further compare RAG-GFM with four state-of-the-art text-free GFMs in terms of fine-tuning efficiency on CiteSeer under **LODO (dataset)**. We report both the number of episodes required to reach stable accuracy and the peak GPU memory usage. As shown in Figure 5, RAG-GFM achieves clear advantages on both fronts. In terms of time, it converges much faster, as retrieval-augmented prompts inject external knowledge directly without costly parameter updates. For memory, most knowledge is externalized to the dual-modal database, where only lightweight prompts are optimized while encoders remain frozen, reducing GPU usage to less than half of MDGFM. Although GCOPE and SAMGPT can reach comparable accuracy, they require 2-3× more episodes and substantially higher memory, which limits their scalability in practice.

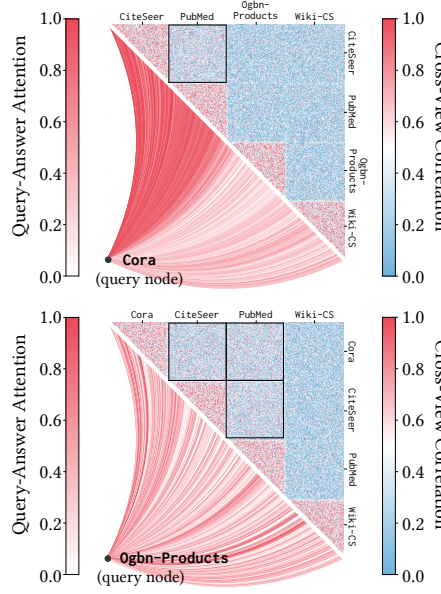


Figure 6: RAG Correlation Map.

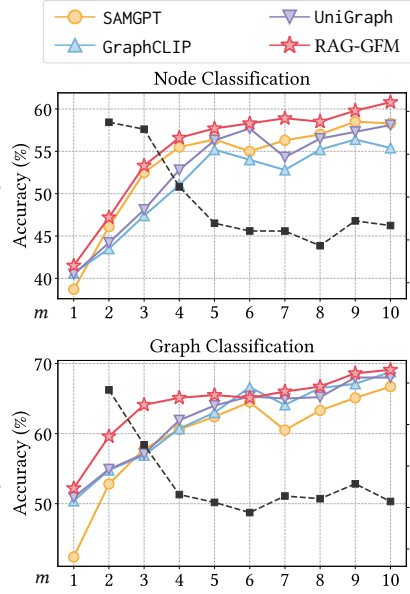
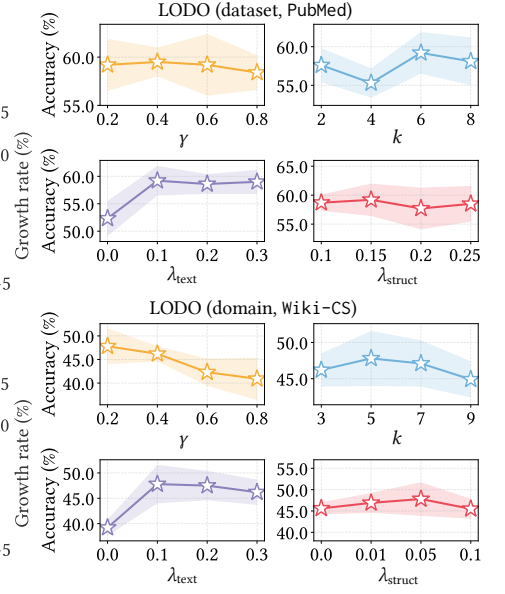
Figure 7: m -Shot Classification (CiteSeer).

Figure 8: Hyper-parameter Analysis.

5.6 RQ5: Reliability and Interpretability of RAG

We assess the reliability and interpretability of RAG by visualizing its retrieval behavior under **LODO (dataset)** and **LODO (domain)** settings. We construct a cross-view correlation map and query-answer attention visualization, where heatmaps capture semantic-structural correlations across source datasets, and curved lines indicate the attention between a query node and retrieved source nodes, with color and thickness reflecting attention intensity.

As shown in Figure 6, clear diagonal blocks emerge in the correlation maps, indicating strong semantic-structural consistency within datasets, while cross-dataset or cross-domain correlations remain low. Datasets from the same domain (e.g., CiteSeer and PubMed) still exhibit relatively higher correlations, suggesting transferable within-domain relations. In **LODO (dataset)**, a query from Cora assigns higher attention to citation-domain sources, reflecting adaptive retrieval of aligned knowledge. In **LODO (domain)**, attention becomes more evenly distributed across unseen domains while maintaining weak but informative focus on partially aligned datasets such as Cora and Wiki-CS.

5.7 RQ6: Sensitivity to Hyper-parameters

We evaluate the robustness of RAG-GFM under two groups.

m -Shot Classification. Figure 7 presents the performance trends as m increases of both node and graph classification tasks on the **LODO (dataset, CiteSeer)** setting. All methods exhibit a saturation curve, where accuracy improves rapidly when moving from extremely low-shot (1-3 samples) to moderate-shot (5-6 samples) and then stabilizes. Notably, RAG-GFM consistently outperforms all baselines at most of the shots, achieving higher accuracy and smoother convergence. The black dashed line depicts its growth rate, showing a sharp improvement at early stages followed by stable gains, indicating that retrieval-augmented prompting accelerates label efficiency and mitigates overfitting in low-shot regimes.

Sensitivity Analysis. Results are shown in Figure 8. Across both **LODO (dataset, PubMed)** and **LODO (domain, Wiki-CS)** settings, performance remains stable under moderate perturbations. For γ , the weight of the domain-token regularizer in Eq. (15), overly large values degrade performance by suppressing domain priors. For k , the number of retrieved query-answer pairs (Section 4.3.2), moderate values best balance retrieval diversity and noise. Accuracy peaks near $\lambda_{\text{text}} = 0.1$ in Eq. (18), indicating its benefits from moderate textual retrieval, while larger values cause drift. Similarly, moderate λ_{struct} in Eq. (19) yields stable performance, as excessive structural signals may introduce bias. Overall, these trends demonstrate robustness without fine-grained tuning.

6 Conclusion

In this work, we propose a Retrieval-Augmented Generation aided Graph Foundation Model named RAG-GFM that mitigates the in-memory bottleneck of existing GFM by externalizing knowledge into a unified semantic-structural retrieval database. Instead of encoding priors into parameters, RAG-GFM decouples parameterized learning from retrievable knowledge, enabling interpretable and efficient adaptation. Through cross-view alignment and retrieval-augmented prompting, the framework achieves efficient generalization across domains and datasets. Extensive experiments demonstrate that RAG-GFM consistently surpasses state-of-the-art GFMs in effectiveness, efficiency, and robustness across diverse settings.

Acknowledgments

The corresponding author is Jianxin Li. Authors of this work are supported in part by NSFC under grants No.623B2010, No.62225202, and No.62302023, by the Fundamental Research Funds for the Central Universities, and by the Academic Excellence Foundation of BUAA for PhD Students. We extend our sincere thanks to all authors for their valuable contributions.

References

- [1] Eric Z Ayers and John T Stasko. 1995. Using graphic history in browsing the World Wide Web. In *WWW*. 451–459.
- [2] Yuxuan Cao, Jiarong Xu, Carl Yang, Jiaan Wang, Yunchao Zhang, Chunping Wang, Lei Chen, and Yang Yang. 2023. When to pre-train graph neural networks? From data generation perspective!. In *KDD*. 142–153.
- [3] Deli Chen, Yankai Lin, Wei Li, Peng Li, Jie Zhou, and Xu Sun. 2020. Measuring and relieving the over-smoothing problem for graph neural networks from the topological view. In *AAAI*, Vol. 34. 3438–3445.
- [4] Ke-Jia Chen, Jiajun Zhang, Linpu Jiang, Yunyun Wang, and Yuxuan Dai. 2022. Pre-training on dynamic graph neural networks. *Neurocomputing* 500 (2022), 679–687.
- [5] Ernesto Estrada and Juan A Rodriguez-Velazquez. 2005. Subgraph centrality in complex networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 71, 5 (2005), 056103.
- [6] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A survey on rag meeting LLMs: Towards retrieval-augmented large language models. In *KDD*. 6491–6501.
- [7] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph neural networks for social recommendation. In *WWW*. 417–426.
- [8] C Lee Giles, Kurt D Bollacker, and Steve Lawrence. 1998. CiteSeer: An automatic citation indexing system. In *Proceedings of the Third ACM Conference on Digital Libraries*. 89–98.
- [9] Vladimir Gligorijević, P Douglas Renfrew, Tomasz Kosciolk, Julia Koehler Leman, Daniel Berenberg, Tommi Vatanen, Chris Chandler, Bryn C Taylor, Ian M Fisk, Hera Vlamakis, et al. 2021. Structure-based protein function prediction using graph convolutional networks. *Nature Communications* 12, 1 (2021), 3168.
- [10] Zihao Guo, Qingyun Sun, Haonan Yuan, Xingcheng Fu, Min Zhou, Yisen Gao, and Jianxin Li. 2025. GraphMoRE: Mitigating topological heterogeneity via mixture of Riemannian experts. In *AAAI*, Vol. 39. 11754–11762.
- [11] Zihao Guo, Qingyun Sun, Ziwei Zhang, Haonan Yuan, Huiping Zhuang, Xingcheng Fu, and Jianxin Li. 2025. GraphKeeper: Graph domain-incremental learning via knowledge disentanglement and preservation. In *NeurIPS*.
- [12] Will Hamilton, Zhitaoying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *NeurIPS* 30 (2017).
- [13] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A Rossi, Subhabrata Mukherjee, Xianfeng Tang, et al. 2024. Retrieval-augmented generation with graphs (GraphRAG). *arXiv preprint arXiv:2501.00309* (2024).
- [14] Yufei He, Yuan Sui, Xiaoxin He, and Bryan Hooi. 2025. UniGraph: Learning a unified cross-domain foundation model for text-attributed graphs. In *KDD*. 448–459.
- [15] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *NeurIPS* 33 (2020), 22118–22133.
- [16] Qian Huang, Hongyu Ren, and Jure Leskovec. 2022. Few-shot relational reasoning via connection subgraph pretraining. *NeurIPS* 35 (2022), 6397–6409.
- [17] Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. In *EMNLP*. 7969–7992.
- [18] Nicolas Keriven. 2022. Not too little, not too much: A theoretical analysis of graph (over) smoothing. *NeurIPS* 35 (2022), 2268–2281.
- [19] Jiho Kim, Sungjin Park, Yeonsu Kwon, Yohan Jo, James Thorne, and Edward Choi. 2023. FactKG: Fact verification via reasoning on knowledge graphs. In *ACL*. 16190–16206.
- [20] Thomas N Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *ICLR*.
- [21] Alexander Kraskov, Harald Stögbauer, and Peter Grassberger. 2004. Estimating mutual information. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 69, 6 (2004), 066138.
- [22] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS* 33 (2020), 9459–9474.
- [23] Xinze Li, Zhenghao Liu, Chenyan Xiong, Shi Yu, Yu Gu, Zhiyuan Liu, and Ge Yu. 2023. Structure-aware language model pretraining improves dense retrieval on structured data. In *ACL Findings*. 11560–11574.
- [24] Yuhao Li, Peisong Wang, Zhixun Li, Jeffrey Xu Yu, and Jia Li. 2024. ZeroG: Investigating cross-dataset zero-shot transferability in graphs. In *KDD*. 1725–1735.
- [25] Hao Liu, Jiarui Feng, Lecheng Kong, Ningyue Liang, Dacheng Tao, Yixin Chen, and Muhan Zhang. 2024. One for all: Towards training one graph model for all classification tasks. In *ICLR*.
- [26] Jingzhe Liu, Haitao Mao, Zhikai Chen, Wenqi Fan, Mingxuan Ju, Tong Zhao, Neil Shah, and Jiliang Tang. 2024. One model for one graph: A new perspective for pretraining with cross-domain graphs. *arXiv preprint arXiv:2412.00315* (2024).
- [27] Zemin Liu, Xingtong Yu, Yuan Fang, and Xinming Zhang. 2023. GraphPrompt: Unifying pre-training and downstream tasks for graph neural networks. In *WWW*. 417–428.
- [28] Yuanfu Lu, Xunqiang Jiang, Yuan Fang, and Chuan Shi. 2021. Learning to pre-train graph neural networks. *AAAI* 35, 5 (2021), 4276–4284.
- [29] Jiayi Luo, Qingyun Sun, Lingjuan Lyu, Ziwei Zhang, Haonan Yuan, Xingcheng Fu, and Jianxin Li. 2026. Towards effective, stealthy, and persistent backdoor attacks targeting graph foundation models. In *AAAI*.
- [30] Jiayi Luo, Qingyun Sun, Yuecen Wei, Haonan Yuan, Xingcheng Fu, and Jianxin Li. 2026. Privacy auditing of multi-domain graph pre-trained model under membership inference attacks. In *AAAI*.
- [31] Linhao Luo, Zicheng Zhao, Gholamreza Haffari, Dinh Phung, Chen Gong, and Shirui Pan. 2025. GFM-RAG: graph foundation model for retrieval augmented generation. In *NeurIPS*.
- [32] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. Query rewriting in retrieval-augmented large language models. In *EMNLP*. 5303–5315.
- [33] Yao Ma and Jiliang Tang. 2021. *Deep learning on graphs*. Cambridge University Press.
- [34] Haitao Mao, Zhikai Chen, Wenzhuo Tang, Jianan Zhao, Yao Ma, Tong Zhao, Neil Shah, Mikhail Galkin, and Jiliang Tang. 2024. Position: Graph foundation models are already here. In *ICML*.
- [35] Andrew Kachites McCallum, Kamal Nigam, Jason Rennie, and Kristie Seymore. 2000. Automating the construction of internet portals with machine learning. *Information Retrieval* 3 (2000), 127–163.
- [36] Péter Mernyei and Cătălina Cangea. 2020. Wiki-CS: A wikipedia-based benchmark for graph neural networks. *arXiv preprint arXiv:2007.02901* (2020).
- [37] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).
- [38] Karl Pearson. 1901. On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* 2, 11 (1901), 559–572.
- [39] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921* (2024).
- [40] Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 2023. In-context retrieval-augmented language models. *TACL* 11 (2023), 1316–1331.
- [41] Vinay Venkatesh Ramasesh, Aitor Lewkowycz, and Ethan Dyer. 2021. Effect of scale on catastrophic forgetting in neural networks. In *ICLR*.
- [42] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. 2008. Collective classification in network data. *AI Magazine* 29, 3 (2008), 93–93.
- [43] Chuan Shi, Junze Chen, Jiawei Liu, and Cheng Yang. 2024. Graph foundation model. *Frontiers of Computer Science* 18, 6 (2024).
- [44] Junhua Shi, Qingyun Sun, Haonan Yuan, and Xingcheng Fu. 2026. SA2GFM: Enhancing robust graph foundation models with structure-aware semantic augmentation. In *AAAI*.
- [45] Joshua Southern, Yam Eitan, Guy Bar-Shalom, Michael M Bronstein, Haggai Maron, and Fabrizio Frasca. 2025. Balancing efficiency and expressiveness: Sub-graph GNNs with walk-based centrality. In *ICML*.
- [46] Fan-Yun Sun, Jordan Hoffman, Vikas Verma, and Jian Tang. 2020. InfoGraph: Un-supervised and semi-supervised graph-level representation learning via mutual information maximization. In *ICLR*.
- [47] Mingchen Sun, Kaixiong Zhou, Xin He, Ying Wang, and Xin Wang. 2022. GPPT: Graph pre-training and prompt tuning to generalize graph neural networks. In *KDD*. 1717–1727.
- [48] Qingyun Sun, Yi Huang, Haonan Yuan, Xingcheng Fu, Yisen Gao, Jia Wu, Shujian Yu, Angsheng Li, Jianxin Li, and Philip S Yu. 2026. Information-Theoretic Foundations and Advances in Graph Machine Learning: A Comprehensive Survey. *Authorea Preprints* (2026).
- [49] Qingyun Sun, Jiaqi Yuan, Shan He, Xiao Guan, Haonan Yuan, Xingcheng Fu, Jianxin Li, and Philip S Yu. 2025. DyG-RAG: Dynamic graph retrieval-augmented generation with event-centric reasoning. *arXiv preprint arXiv:2507.13396* (2025).
- [50] Bosiljka Tadić. 2001. Dynamics of directed graphs: the world-wide Web. *Physica A: Statistical Mechanics and its Applications* 293, 1-2 (2001), 273–284.
- [51] Yanchao Tan, Zihao Zhou, Hang Lv, Weiming Liu, and Carl Yang. 2024. WalkLM: A uniform language model fine-tuning framework for attributed graph embedding. *NeurIPS* 36 (2024).
- [52] Jiabin Tang, Yuhao Yang, Wei Wei, Lei Shi, Long Xia, Dawei Yin, and Chao Huang. 2024. HiGPT: Heterogeneous graph language model. In *KDD*. 2842–2853.
- [53] Wenzhuo Tang, Haitao Mao, Danial Dervovic, Ivan Brugere, Saumitra Mishra, Yuying Xie, and Jiliang Tang. 2024. Cross-domain graph data scaling: a showcase with diffusion models. *arXiv preprint arXiv:2406.01899* (2024).
- [54] Yixuan Tang and Yi Yang. 2024. Multihop-RAG: Benchmarking retrieval-augmented generation for multi-hop queries. *arXiv preprint arXiv:2401.15391* (2024).
- [55] Qwen Team et al. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671* 2 (2024), 3.

- [56] Yijun Tian, Huan Song, Zichen Wang, Haozhu Wang, Ziqing Hu, Fang Wang, Nitesh V Chawla, and Panpan Xu. 2024. Graph neural prompting with large language models. In *AAAI*, Vol. 38. 19080–19088.
- [57] Naftali Tishby, Fernando C Pereira, and William Bialek. 2000. The information bottleneck method. *arXiv preprint physics/0004057* (2000).
- [58] Naftali Tishby and Noga Zaslavsky. 2015. Deep learning and the information bottleneck principle. In *IEEE Information Theory Workshop*. IEEE, 1–5.
- [59] Petar Velićković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph attention networks. In *ICLR*.
- [60] Petar Velićković, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. 2019. Deep Graph Infomax. In *ICLR*.
- [61] Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. 2024. Can language models solve graph problems in natural language? *NeurIPS* 36 (2024).
- [62] Hongwei Wang, Fuzheng Zhang, Mengdi Zhang, Jure Leskovec, Miao Zhao, Wenjie Li, and Zhongyuan Wang. 2019. Knowledge-aware graph neural networks with label smoothness regularization for recommender systems. In *KDD*. 968–977.
- [63] Quan Wang, Zhendong Mao, Bin Wang, and Li Guo. 2017. Knowledge graph embedding: A survey of approaches and applications. *IEEE TKDE* 29, 12 (2017), 2724–2743.
- [64] Shuo Wang, Bokui Wang, Zhixiang Shen, Boyan Deng, and Zhao Kang. 2025. Multi-domain graph foundation models: robust knowledge transfer via topology alignment. *ICML* (2025).
- [65] Shu Wu, Yuyuan Tang, Yangqiao Zhu, Liang Wang, Xing Xie, and Tieniu Tan. 2019. Session-based recommendation with graph neural networks. *AAAI* 33, 01 (2019), 346–353.
- [66] Feng Xia, Ke Sun, Shuo Yu, Abdul Aziz, Liangtian Wan, Shirui Pan, and Huan Liu. 2021. Graph learning: A survey. *IEEE TAI* 2, 2 (2021), 109–127.
- [67] Kewei Xiong, Wei Wang, Ruofan Ding, Dinglin Luo, Yangmei Qin, Xudong Zou, Jiguang Wang, Chen Yu, and Lei Li. 2026. Multimodal-based analysis of single-cell ATAC-seq data enables highly accurate delineation of clinically relevant tumor cell subpopulations. *Genome Medicine* (2026).
- [68] Yaming Yang, Ziyu Guan, Zhe Wang, Wei Zhao, Cai Xu, Weigang Lu, and Jianbin Huang. 2022. Self-supervised heterogeneous graph pre-training based on structural clustering. *NeurIPS* 35 (2022), 16962–16974.
- [69] Gustavo Ye. 2024. nano-vectordb. <https://github.com/gusye1234/nano-vectordb>
- [70] Zixuan Yi, Iadh Ounis, and Craig Macdonald. 2023. Contrastive graph prompt-tuning for cross-domain recommendation. *ACM TOIS* 42, 2 (2023), 1–28.
- [71] Jun Yin, Chaozhao Li, Hao Yan, Jianxun Lian, and Senzhang Wang. 2023. Train once and explain everywhere: Pre-training interpretable graph neural networks. *NeurIPS* 36 (2023), 35277–35299.
- [72] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph contrastive learning with augmentations. *NeurIPS* 33 (2020), 5812–5823.
- [73] Xingtong Yu, Yuan Fang, Zemin Liu, and Xinming Zhang. 2024. HGPrompt: Bridging homogeneous and heterogeneous graphs for few-shot prompt learning. *AAAI* 38, 15 (2024), 16578–16586.
- [74] Xingtong Yu, Zechuan Gong, Chang Zhou, Yuan Fang, and Hui Zhang. 2025. Samgpt: Text-free graph foundation model for multi-domain pre-training and cross-domain adaptation. In *WWW*. 1142–1153.
- [75] Xingtong Yu, Chang Zhou, Yuan Fang, and Xinming Zhang. 2024. Text-free multi-domain graph pre-training: toward graph foundation models. *arXiv preprint arXiv:2405.13934* (2024).
- [76] Haonan Yuan, Qingyun Sun, Xingcheng Fu, Ziwei Zhang, Cheng Ji, Hao Peng, and Jianxin Li. 2023. Environment-aware dynamic graph learning for out-of-distribution generalization. *NeurIPS* 36 (2023), 49715–49747.
- [77] Haonan Yuan, Qingyun Sun, Junhua Shi, Xingcheng Fu, Bryan Hooi, Jianxin Li, and Philip S Yu. 2025. GRAVER: Generative graph vocabularies for robust graph foundation models fine-tuning. In *NeurIPS*.
- [78] Haonan Yuan, Qingyun Sun, Junhua Shi, Xingcheng Fu, Bryan Hooi, Jianxin Li, and Philip S Yu. 2025. How much can transfer? BRIDGE: Bounded multi-domain graph foundation model with generalization guarantees. In *ICML*.
- [79] Yongqi Zhang and Quanming Yao. 2022. Knowledge graph reasoning with relational digraph. In *WWW*. 912–924.
- [80] Zaixi Zhang, Qi Liu, Hao Wang, Chengqiang Lu, and Chee-Kong Lee. 2021. Motif-based graph self-supervised learning for molecular property prediction. *NeurIPS* 34 (2021), 15870–15882.
- [81] Haihong Zhao, Aochuan Chen, Xiangguo Sun, Hong Cheng, and Jia Li. 2024. All in one and one for all: A simple yet effective method towards cross-domain graph pretraining. In *KDD*. 4443–4454.
- [82] Hongbo Zhao, Bolin Ni, Junsong Fan, Yuxi Wang, Yuntao Chen, Gaofeng Meng, and Zhaoxiang Zhang. 2024. Continual forgetting for pre-trained vision models. In *CVPR*. 28631–28642.
- [83] Yun Zhu, Haizhou Shi, Xiaotang Wang, Yongchao Liu, Yaoke Wang, Boci Peng, Chuntao Hong, and Siliang Tang. 2025. GraphCLIP: Enhancing transferability in graph foundation models for text-attributed graphs. In *WWW*. 2183–2197.

A Algorithms and Complexity Analysis

We illustrate the overall pre-training pipeline of RAG-GFM in Algorithm 1, and the fine-tuning pipeline in Algorithm 2.

A.1 Complexity Analysis of Algorithm 1

The pre-training pipeline mainly consists of three stages:

Database Construction. For each source domain D_i^S with N_i nodes, E_i edges, and feature dimension d_i , we first project node features into a unified space of dimension d_0 via PCA, which costs $O(N_i d_i d_0)$. BERT encodes each node’s text with the complexity of $O(N_i C_{\text{BERT}})$, where C_{BERT} denotes the per-sample encoding cost. For the structural store, computing K -order WSE requires $O(KE_i)$ with sparse matrix multiplications, followed by top- M_i anchor selection $O(N_i \log N_i)$ and h -hop ego-subgraph extraction $O(M_i \bar{d}_i^h)$ with $\bar{d}_i$ as average degree. Over n domains, the complexity is:

$$O\left(\sum_{i=1}^n [N_i d_i d_0 + N_i C_{\text{BERT}} + KE_i + N_i \log N_i + M_i \bar{d}_i^h]\right). \quad (\text{A.1})$$

Cross-view Encoding. At each iteration, a mixed-domain batch $\mathcal{B}$ of size $|\mathcal{B}|$ is sampled. Two L -layer GNN encoders with dimension d are applied to semantic and structural views, giving complexity:

$$O(2L(E_{\mathcal{B}} + |\mathcal{B}|)d), \quad (\text{A.2})$$

where $E_{\mathcal{B}}$ is the number of edges in the sampled batch $\mathcal{B}$.

Self-supervised Pre-training. The cross-view InfoNCE computes pairwise similarities, with cost $O(|\mathcal{B}|^2 d)$. The compression regularizers introduce $O(|\mathcal{B}|d)$, negligible compared to the quadratic term. Token regularization across n domains costs $O(nd)$.

Overall Complexity. The dominant cost per iteration is:

$$O(L(E_{\mathcal{B}} + |\mathcal{B}|)d + |\mathcal{B}|^2 d), \quad (\text{A.3})$$

while database construction is a one-time preprocessing overhead.

Summary. The dominant cost comes from GNN propagation and quadratic contrastive alignment. Database construction is performed once and is negligible compared to iterative training.

A.2 Complexity Analysis of Algorithm 2

In the fine-tuning phase, the encoders Θ_t^* , Θ_s^* and domain tokens $\{\tau_D\}$ are frozen, and only prompt parameters Ω are optimized.

Preprocessing. For m -shot support instances with raw dimension d^T , preprocessing requires $O(md^T d_0 + mC_{\text{BERT}})$.

Domain-gated Fusion. For each support instance, similarities with n domain tokens are computed at cost $O(mnd)$.

Semantic Retrieval. Each query searches $\mathcal{D}_{\text{text}}$ of size M_{text} using approximate nearest neighbor (ANN) with $O(\log M_{\text{text}})$ per query. Aggregating top- k answers incurs $O(kd_0)$. The total cost is:

$$O(m(\log M_{\text{text}} + kd_0)). \quad (\text{A.4})$$

Structural Retrieval. Each query searches the structural store of n domains, each of size M_{struct} . ANN search costs $O(n \cdot \log M_{\text{struct}})$, and fusing motif features requires $O(nd)$. Thus:

$$O(m(n \cdot \log M_{\text{struct}} + nd)). \quad (\text{A.5})$$

Prompted adaptation. Prompt construction and concatenation cost $O(md)$. The InfoNCE fine-tuning loss requires similarity with C class prototypes, giving $O(mCd)$.

Algorithm 1: Overall pre-training pipeline of RAG-GFM.

Input: n source graphs $\{G_i^S\}_{i=1}^n$ from domain $\{D^S\}$; Batch size $\mathcal{B}$; Learning rate η_1 ; Pre-training epochs E_1 .
Output: Graph learner $h = g \circ f$ with parameters Θ_t^* and Θ_s^* ; Domain tokens $\{\tau_{D_i}\}_{i=1}^n$.

- 1 Initialize all learnable parameters randomly;
- 2 // Establish the Unified Retrieval Database
- 3 **for** each G_i^S in $\{G_i^S\}_{i=1}^n$ **do**
- 4 Representation track: $Z_i^S \leftarrow \text{Eq. (2), Eq. (3)}$;
- 5 Retrieval track: $\tilde{z}_v^S \leftarrow \text{Eq. (4)}$ for each node v ;
- 6 Establish the semantic store: $\mathcal{D}_{\text{text}} \leftarrow \text{Eq. (4)}$;
- 7 Establish the structural store: $\mathcal{D}_{\text{struct}} \leftarrow \text{Eq. (8)}$;
- 8 // Compose Node Views
- 9 **for** $e_1 = 1, 2, \dots, E_1$ **do**
- 10 **for** each G_i^S in $\{G_i^S\}_{i=1}^n$ **do**
- 11 Learn node semantic view: $Z_i^S \leftarrow \text{Eq. (2), Eq. (3)}$;
- 12 Learn node structural view: $W_i^S \leftarrow \text{Eq. (9)}$;
- 13 // Self-supervised Information Bottleneck
- 14 Encode dual-embeddings: $H_i^{\text{text}}, H_i^{\text{struct}} \leftarrow \text{Eq. (11)}$;
- 15 **for** each node v in the sampled batch $\mathcal{B}$ **do**
- 16 Calculate the alignment loss: $\mathcal{L}_{\text{align}}^{(i,v)} \leftarrow \text{Eq. (12)}$;
- 17 // Token Regularization and Parameter Update
- 18 Calculate overall pre-training loss: $\mathcal{L}_{\text{pretrain}} \leftarrow \text{Eq. (15)}$;
- 19 Update the model parameters Θ_t, Θ_s by minimizing $\mathcal{L}_{\text{pretrain}}$ and back-propagation with learning rate η_1 ;

Overall complexity. The dominant cost per iteration is:

$$O(m[\log M_{\text{text}} + n \cdot \log M_{\text{struct}} + (k + n + C)d]). \quad (\text{A.6})$$

Summary. The main bottleneck lies in retrieval (logarithmic in database size) and classification overhead. Since the backbone parameters are frozen and only lightweight prompts are updated, fine-tuning is substantially more efficient than full adaptation.

B Proofs

B.1 Proof of Proposition 1

We first restate Proposition 1 for reference.

Proposition 1 (Structural Separability of WSE). There exist pairs of non-isomorphic graphs G_1, G_2 and nodes $v \in G_1, u \in G_2$ such that for any fixed radius r , the r -hop neighbors $\mathcal{N}_r(v)$ and $\mathcal{N}_r(u)$ are isomorphic, yet the Walk-Spectrum Encodings satisfy:

$$\mathbf{C}_\alpha^{\text{WSE}}(v) \neq \mathbf{C}_\alpha^{\text{WSE}}(u).$$

PROOF. The sketch is to construct a pair of graphs that are cospectral locally (same r -neighbor) but differ in global cycle structure (e.g., attaching different length cycles far from the root while preserving the first r shells). Closed-walk counts at the root incorporate returns that traverse those distant cycles, which appear only at higher orders. Hence, a finite K separates v and u . This ensures

Algorithm 2: Overall fine-tuning pipeline of RAG-GFM.

Input: Unified database $\mathcal{D}$; Target domain D^T ; Target graph(s) and m -shot support set $\mathcal{S}^T$; Frozen parameters Θ_t^* and Θ_s^* ; Frozen domain tokens $\{\tau_{D_i}\}_{i=1}^n$; Learning rate η_2 ; Fine-tuning epochs E_2 .
Output: Fine-tuned graph learner $h^* = g^*(f^*)$ with parameters $\{\Theta_t^*, \Theta_s^*, \Omega^*\}$.

- 1 Initialize all learnable parameters randomly;
- 2 // Preprocess Support Set
- 3 **for** each support node (or graph) in $\mathcal{S}^T$ **do**
- 4 Learn dimension-aligned feature $\hat{\mathbf{X}}^T \leftarrow \text{Eq. (2), Eq. (3)}$;
- 5 **for** $e_1 = 1, 2, \dots, E_2$ **do**
- 6 **for** each support node (or graph) in $\mathcal{S}^T$ **do**
- 7 Encode via pre-trained learner: $Z_i^T \leftarrow \text{Eq. (16)}$;
- 8 // Domain-gated Fusion
- 9 Calculate gating weights: $\{\pi_{i,k}\}_{k=1}^n \leftarrow \text{Eq. (16)}$;
- 10 // Semantic Query and Retrieval
- 11 Query $\mathcal{D}_{\text{text}}$ and get answers: $(\Delta z_v^T)^{\text{text}} \leftarrow \text{Eq. (17)}$;
- 12 Query $\mathcal{D}_{\text{struct}}$ and get answers: $(\Delta z_v^T)^{\text{struct}} \leftarrow \text{Eq. (20)}$;
- 13 // In-context Augmentation and Prompt
- 14 Update instance embedding: $z_v^{T''} \leftarrow \text{Eq. (18), Eq. (19)}$;
- 15 Initialize $P_\Omega \leftarrow \text{Eq. (21)}$ and prompt: $\mathbf{h}_i^T \leftarrow \text{Eq. (21)}$;
- 16 // Few-shot Adaptation and Parameter Update
- 17 Calculate the fine-tuning loss: $\mathcal{L}_{\text{fine-tune}} \leftarrow \text{Eq. (22)}$;
- 18 Update the prompt parameters Ω by minimizing $\mathcal{L}_{\text{fine-tune}}$ and back-propagation with learning rate η_2 ;

that traditional r -hop methods [45] cannot distinguish them, while WSE produces different signatures.

Formally, fix any radius r . Let $P_{r+1} = (x_0, x_1, \dots, x_{r+1})$ be a path of length $r+1$, with root x_0 . At the endpoint x_{r+1} , we attach a cycle.

In graph G_1 , we attach an odd cycle C_p of length $p \geq 3$, while in graph G_2 , we attach an even cycle C_q of length $q \geq 4$. Denote the roots by $v = x_0 \in G_1$ and $u = x_0 \in G_2$.

As the cycle appears only beyond radius r , the neighbors $\mathcal{N}_r(v)$ and $\mathcal{N}_r(u)$ both reduce to path $(x_0, \dots, x_r)$, and are isomorphic:

$$\mathcal{N}_r(v) \cong \mathcal{N}_r(u). \quad (\text{B.1})$$

Consider closed walks that go from the root to the cycle, traverse it, and return. In G_1 , the shortest such closed walk has length $K_1 = 2(r+1) + p$, while in G_2 the shortest length is $K_2 = 2(r+1) + q$, and in fact any closed walk using the cycle in G_2 has length:

$$K = 2(r+1) + q + 2\ell, \quad \ell \geq 0, \quad (\text{B.2})$$

which is always even. Since p is odd, K_1 is odd, and therefore:

$$(\mathbf{A}_{G_1}^{K_1})_{vv} > 0, \quad (\mathbf{A}_{G_2}^{K_1})_{uu} = 0. \quad (\text{B.3})$$

By the definition of WSE that $\mathbf{C}_\alpha^{\text{WSE}}(z)[k] = \alpha^k \mathbf{A}_{zz}^k$, so the two encodings must differ at coordinate K_1 . Hence, we conclude that:

$$\mathbf{C}_\alpha^{\text{WSE}}(v) \neq \mathbf{C}_\alpha^{\text{WSE}}(u), \quad (\text{B.4})$$

which establishes that WSE separates nodes indistinguishable by local neighborhoods. We conclude the proof. $\square$

B.2 Proof of Proposition 2

We first restate Proposition 2 for reference.

Proposition 2 (Cross-View Mutual Information Bounds). The relevance term admits the InfoNCE lower-bound estimator:

$$I(\mathbf{h}_{i,v}^{\text{text}}; \mathbf{h}_{i,v}^{\text{struct}}) \leq \frac{1}{|\mathcal{B}|} \sum_{v \in \mathcal{B}} \log \frac{\exp(\sigma(g_t(\mathbf{h}_{i,v}^{\text{text}}), g_s(\mathbf{h}_{i,v}^{\text{struct}}))/\tau)}{\sum_{u \in \mathcal{B}} \exp(\sigma(g_t(\mathbf{h}_{i,v}^{\text{text}}), g_s(\mathbf{h}_{i,u}^{\text{struct}}))/\tau)},$$

where g_t, g_s are projections, $\sigma(\cdot)$ is similarity, τ is a temperature, positives are formed by the same node across the views (v, v) in a batch $\mathcal{B}$, and negatives by mismatched nodes $(v, u), u \neq v$.

The compression term can be upper-bounded via KL-divergence:

$$I(\mathbf{h}_{i,v}^{\text{text}}; \bar{\mathbf{x}}_{i,v}^S) \geq \mathbb{E}_{p(\mathbf{h}_{i,v}^{\text{text}}, \bar{\mathbf{x}}_{i,v}^S)} [\log q_\phi(\mathbf{h}_{i,v}^{\text{text}} | \bar{\mathbf{x}}_{i,v}^S)] - \mathbb{E}_{p(\mathbf{h}_{i,v}^{\text{text}})} [\log p(\mathbf{h}_{i,v}^{\text{text}})],$$

where v is sampled from $\mathcal{B}$, $\mathbf{x}$ denotes $\mathbf{z}$ or $\mathbf{w}$, and $q_\phi(\cdot|\cdot)$ is a variational approximation of the true conditional distribution.

PROOF. (1) Relevance Term (InfoNCE Lower Bound). The mutual information between semantic and structural embeddings is:

$$I(\mathbf{h}_{i,v}^{\text{text}}; \mathbf{h}_{i,v}^{\text{struct}}) = \mathbb{E}_{p(\mathbf{h}_{i,v}^{\text{text}}, \mathbf{h}_{i,v}^{\text{struct}})} \left[\log \frac{p(\mathbf{h}_{i,v}^{\text{text}} | \mathbf{h}_{i,v}^{\text{struct}})}{p(\mathbf{h}_{i,v}^{\text{text}})} \right]. \quad (\text{B.5})$$

Directly computing is intractable. Following the contrastive estimation framework of InfoNCE [37], we approximate it with a similarity-based classification task that distinguishes positive pairs (same node across views) from negative pairs (different nodes).

Given a batch $\mathcal{B}$ of nodes, we define the similarity score $s_{vu} = \sigma(g_t(\mathbf{h}_{i,v}^{\text{text}}), g_s(\mathbf{h}_{i,u}^{\text{struct}}))$, where g_t, g_s are projection heads and $\sigma(\cdot)$ is similarity. Then the InfoNCE lower bound becomes:

$$I(\mathbf{h}_{i,v}^{\text{text}}; \mathbf{h}_{i,v}^{\text{struct}}) \geq \frac{1}{|\mathcal{B}|} \sum_{v \in \mathcal{B}} \log \frac{\exp(s_{vv}/\tau)}{\sum_{u \in \mathcal{B}} \exp(s_{vu}/\tau)}, \quad (\text{B.6})$$

where τ is a temperature. This corresponds to Eq. (13) and provides a variational lower bound of the cross-view relevance, encouraging semantic and structural embeddings of the same node to align while contrasting mismatched pairs.

(2) Compression Term (KL Upper Bound). We consider the mutual information between an embedding $\mathbf{h}_{i,v}^S$ (either semantic or structural) and its input feature $\mathbf{X}_i^S$. By definition:

$$\begin{aligned} I(\mathbf{h}_{i,v}^S; \mathbf{X}_i^S) &= \mathbb{E}_{p(\mathbf{h}, \mathbf{X})} \left[\log \frac{p(\mathbf{h} | \mathbf{X})}{p(\mathbf{h})} \right] = \mathbb{E}_{p(\mathbf{h}, \mathbf{X})} [\log p(\mathbf{h} | \mathbf{X})] \\ &= \mathbb{E}_{p(\mathbf{h})} [\log p(\mathbf{h})]. \end{aligned} \quad (\text{B.7})$$

Since the conditional posterior $p(\mathbf{h} | \mathbf{X})$ is unknown, we introduce a variational approximation $q_\phi(\mathbf{h} | \mathbf{X})$. Using the non-negativity of the KL divergence, we obtain an upper bound:

$$\mathbb{E}_{p(\mathbf{h}, \mathbf{X})} [\log p(\mathbf{h} | \mathbf{X})] \leq \mathbb{E}_{p(\mathbf{h}, \mathbf{X})} [\log q_\phi(\mathbf{h} | \mathbf{X})]. \quad (\text{B.8})$$

Substituting Eq. (B.8) into Eq. (B.7) gives:

$$I(\mathbf{h}_{i,v}^S; \mathbf{X}_i^S) \leq \mathbb{E}_{p(\mathbf{h}, \mathbf{X})} [\log q_\phi(\mathbf{h} | \mathbf{X})] = \mathbb{E}_{p(\mathbf{h})} [\log p(\mathbf{h})], \quad (\text{B.9})$$

which corresponds to Eq. (14). This upper bound serves as a variational surrogate that penalizes redundant signals while maintaining tractability. The first term encourages compression through reconstruction under q_ϕ , and the second term regularizes the marginal entropy of the latent representation. $\square$

Table C.1: Statistics of the multi-domain graph dataset.

Dataset	Domain	#Node	#Edge	#Feat. Dim.	#Class	Avg. #Deg.
Cora [35]	Citation	2,708	5,429	1,433	7	4.00
CiteSeer [8]	Citation	3,186	4,277	3,703	6	2.57
PubMed [42]	Citation	19,717	44,338	500	3	4.50
Ogbn-Products (Tech.) [15]	E-Commerce	47,428	2,077,241	100	3	87.60
Ogbn-Products (Home) [15]	E-Commerce	9,790	131,841	100	5	26.93
Wiki-CS [36]	Web Link	11,701	216,123	300	10	36.94

C Experiment Details

C.1 Dataset Details

- **Citation Domain:** Cora [35], CiteSeer [8], and PubMed [42], where nodes represent papers and edges denote citation links. Each node is equipped with text-based features derived from titles or abstracts.
- **E-Commerce Domain:** contains two subgraphs from the large-scale Ogbn-Products [15], including Ogbn-Tech and Ogbn-Home. Nodes represent Amazon products, edges indicate co-purchase relationships, and node labels correspond to product categories, capturing consumer behavior.
- **Web Link Domain:** consists of the Wiki-CS [36] dataset, where nodes correspond to Wikipedia articles and edges represent hyperlinks. Textual embeddings extracted from article content provide rich semantic information for web-scale graph learning.

C.2 Implementation Details

We introduce the general implementation details below.

Pre-training. We pre-train RAG-GFM for up to 10,000 epochs with early stopping for 50 consecutive epochs. Both semantic and structural encoders are 2-layer GNNs. The overall pre-training objective combines the cross-view information bottleneck loss and the domain-token regularizer weighted by γ tuned within the range of $[0, 1]$. The Adam optimizer is adopted, with the learning rate and weight decay selected from $[10^{-5}, 10^{-1}]$ via grid search on the validation set. All parameters are initialized from scratch.

Fine-tuning. We fine-tune the RAG-GFM for up to 100 episodes with an early stopping strategy. The pretrained encoder parameters are frozen, and only the prompt parameters are updated. For each query node, we retrieve k ($[1, 10]$) query-answer pairs from both the semantic and structural databases, which are fused with weights λ_{text} ($[0, 1]$) and λ_{struct} ($[0, 1]$) to form retrieval-augmented prompts. The final fine-tuning objective is optimized by Adam with the same learning rate and weight decay settings as in pre-training.

Environment. Experiments are conducted with:

- Operating System: Ubuntu 20.04 LTS.
- CPU: Intel(R) Xeon(R) Platinum 8358 CPU@2.60GHz with 1TB DDR4 of Memory.
- GPU: NVIDIA Tesla A100 SMX4 with 80GB of Memory.
- Software: CUDA 10.1, Python 3.8.12, PyTorch 1.9.1, PyTorch Geometric 2.0.1, NanoVectorDB 0.0.4.3.